

A data analyst wants to group numerical values into clusters. Write a Python program to implement K-Means clustering with two clusters using the dataset and display final cluster assignments

```
File Edit Format Run Options Window Help
data = [2, 4, 6, 8]

centroids = [2.0, 8.0]

for _ in range(10):
    cluster1 = []
    cluster2 = []

    for point in data:
        d1 = abs(point - centroids[0])
        d2 = abs(point - centroids[1])

        if d1 < d2:
            cluster1.append(point)
        else:
            cluster2.append(point)

    new_c1 = sum(cluster1) / len(cluster1) if cluster1 else centroids[0]
    new_c2 = sum(cluster2) / len(cluster2) if cluster2 else centroids[1]

    if centroids == [new_c1, new_c2]:
        break

    centroids = [new_c1, new_c2]

print("Final Centroids:", centroids)
print("Cluster 1:", cluster1)
print("Cluster 2:", cluster2)
```

```
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/saimo/OneDrive/ドキュメント/Desktop/lab model Q1.py =====
Final Centroids: [3.0, 7.0]
Cluster 1: [2, 4]
Cluster 2: [6, 8]
>>>
```

Using the same dataset, write a Python program to implement the Expectation Maximization algorithm for Gaussian mixture clustering. Display updated parameters and final cluster memberships.

```
lab model Q2.py - C:/Users/saimo/OneDrive/ドキュメント/Desktop/lab model Q2.py (3.13.5)
File Edit Format Run Options Window Help
data = [2, 4, 6, 8]

centroids = [2.0, 8.0]

for _ in range(10):
    cluster1 = []
    cluster2 = []

    for point in data:
        d1 = abs(point - centroids[0])
        d2 = abs(point - centroids[1])

        if d1 < d2:
            cluster1.append(point)
        else:
            cluster2.append(point)

    new_c1 = sum(cluster1) / len(cluster1) if cluster1 else centroids[0]
    new_c2 = sum(cluster2) / len(cluster2) if cluster2 else centroids[1]

    if centroids == [new_c1, new_c2]:
        break

    centroids = [new_c1, new_c2]

print("Final Centroids:", centroids)
print("Cluster 1:", cluster1)
print("Cluster 2:", cluster2)
```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/saimo/OneDrive/ドキュメント/Desktop/lab model Q2.py =====
Final Centroids: [3.0, 7.0]
Cluster 1: [2, 4]
Cluster 2: [6, 8]
>>>
```

Compare clustering results obtained from K-Means and EM algorithms using Python outputs. Explain differences in cluster assignments and discuss sensitivity to initial parameter selection.

```

File Edit Format Run Options Window Help
# -----
import math

means = [2.0, 8.0]
variances = [1.0, 1.0]
weights = [0.5, 0.5]

def gaussian(x, mean, var):
    return (1 / math.sqrt(2 * math.pi * var)) * math.exp(-(x - mean) ** 2 / (2 * var))

for _ in range(10):
    responsibilities = []

    for x in data:
        p1 = weights[0] * gaussian(x, means[0], variances[0])
        p2 = weights[1] * gaussian(x, means[1], variances[1])
        total = p1 + p2

        responsibilities.append((p1 / total, p2 / total))

    for k in range(2):
        resp_sum = sum(r[k] for r in responsibilities)

        means[k] = sum(r[k] * data[i] for i, r in enumerate(responsibilities)) / resp_sum
        variances[k] = sum(r[k] * (data[i] - means[k])**2 for i, r in enumerate(responsibilities)) / resp_sum
        weights[k] = resp_sum / len(data)

print("\nEM Algorithm Result:")
print("Means:", means)
print("Variances:", variances)
print("Weights:", weights)

print("\nResponsibilities (Probabilities):")
for i, r in enumerate(responsibilities):
    print(f"Data {data[i]} -> Cluster1: {round(r[0],2)}, Cluster2: {round(r[1],2)}")

# -----
# COMPARISON OUTPUT
# -----
print("\nComparison:")
print("K-Means: Hard clustering (exact groups)")
print("EM: Soft clustering (probabilistic assignment)")

print("\nObservation:")
print("K-Means clusters: [2,4] and [6,8]")
print("EM shows probabilities, middle points belong partially to both clusters")

```

```

IDLE Shell 3.13.5
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/saimo/OneDrive/ドキュメント/Desktop/lab model
q3.py =====
K-Means Result:
Centroids: [3.0, 7.0]
Cluster 1: [2, 4]
Cluster 2: [6, 8]

EM Algorithm Result:
Means: [3.0280176400907717, 6.971982359909228]
Variances: [1.1112855722068304, 1.1112855722068302]
Weights: [0.5, 0.49999999999999994]

Responsibilities (Probabilities):
Data 2 -> Cluster1: 1.0, Cluster2: 0.0
Data 4 -> Cluster1: 0.97, Cluster2: 0.03
Data 6 -> Cluster1: 0.03, Cluster2: 0.97
Data 8 -> Cluster1: 0.0, Cluster2: 1.0

Comparison:
K-Means: Hard clustering (exact groups)
EM: Soft clustering (probabilistic assignment)

Observation:
K-Means clusters: [2,4] and [6,8]
EM shows probabilities, middle points belong partially to both clusters
>>>

```

Analyze and compare convergence behavior of K-Means and EM algorithms. Use program outputs to explain stability, iteration count, and the effect of initialization on final clustering results.

```

File Edit Format Run Options Window Help
data = [2, 4, 6, 8]
centroids = [2.0, 8.0]
kmeans_iterations = 0
for _ in range(10):
    kmeans_iterations += 1
    cluster1 = []
    cluster2 = []

    for point in data:
        d1 = abs(point - centroids[0])
        d2 = abs(point - centroids[1])

        if d1 < d2:
            cluster1.append(point)
        else:
            cluster2.append(point)

    new_c1 = sum(cluster1) / len(cluster1) if cluster1 else centroids[0]
    new_c2 = sum(cluster2) / len(cluster2) if cluster2 else centroids[1]

    if centroids == [new_c1, new_c2]:
        break

    centroids = [new_c1, new_c2]

print("K-Means Convergence:")
print("Iterations:", kmeans_iterations)
print("Final Centroids:", centroids)
print("Clusters:", cluster1, cluster2)

import math

means = [2.0, 8.0]
variances = [1.0, 1.0]
weights = [0.5, 0.5]
em_iterations = 0

def gaussian(x, mean, var):
    return (1 / math.sqrt(2 * math.pi * var)) * math.exp(-(x - mean) ** 2 / (2 * var))

for _ in range(10):
    em_iterations += 1
    responsibilities = []

    for x in data:
        p1 = weights[0] * gaussian(x, means[0], variances[0])
        p2 = weights[1] * gaussian(x, means[1], variances[1])

```

```

IDLE Shell 3.13.5
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/saimo/OneDrive/ドキュメント/Desktop/lab model
q4.py =====
K-Means Convergence:
Iterations: 2
Final Centroids: [3.0, 7.0]
Clusters: [2, 4] [6, 8]

EM Algorithm Convergence:
Iterations: 10
Final Means: [3.0280176400907717, 6.971982359909228]
Variances: [1.1112855722068304, 1.1112855722068302]

Convergence Analysis:
K-Means:
- Converges faster (few iterations)
- Sensitive to initial centroids
- May reach local optimum

EM Algorithms:
- Takes more iterations
- Converges smoothly using probabilities
- Sensitive to initial parameters

Conclusion:
K-Means is faster but EM is more stable and flexible.
>>>

```